

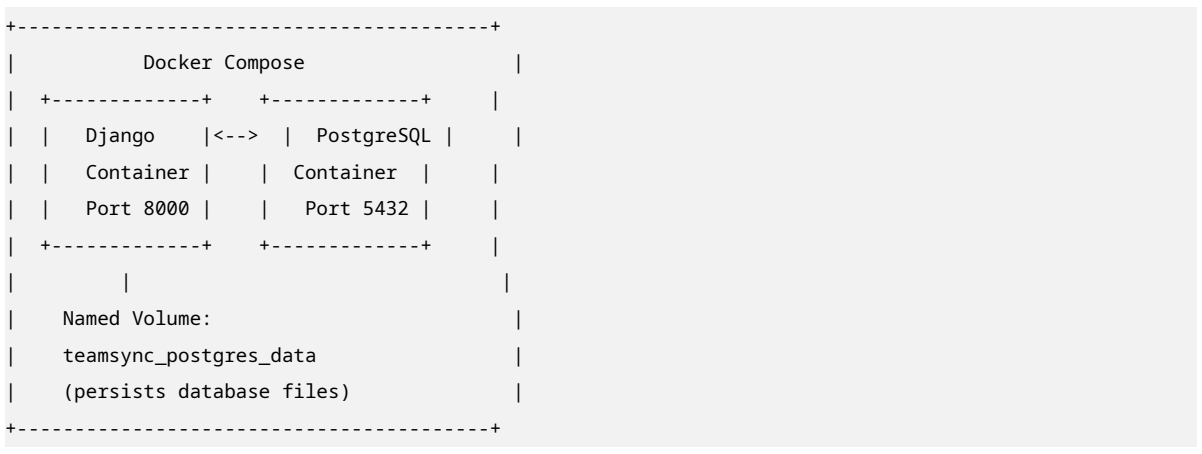
TeamSync Docker Setup — Complete Troubleshooting Guide

Version: 1.0 **Date:** August 2026 **Project:** TeamSync (Django + PostgreSQL + Docker) **Purpose:** Step-by-step guide + all issues faced + exact solutions

Table of Contents

- 1. Architecture Overview
- 2. Prerequisites
- 3. File-by-File Setup
- 4. Running the Project
- 5. All Issues Faced & Solutions
- 6. Common Commands
- 7. For Future AI Agents / Developers

1. Architecture Overview



Service	Image	Port	Purpose
web	Python 3.12 slim	8000	Django application
db	PostgreSQL 16 Alpine	5432	Database

2. Prerequisites

- Docker Desktop installed and running
- Windows PowerShell or Command Prompt
- Project structure is FLAT (no backend/ folder — everything at teamsync/ root)

3. File-by-File Setup

3.1 requirements.txt

```
Django>=5.0
django-rest-framework>=3.14
django-rest-framework-simplejwt>=5.3
django-filter>=23.5
drf-spectacular>=0.27
python-dotenv>=1.0
django-cors-headers>=4.3
psycopg2-binary>=2.9
```

CRITICAL: psycopg2-binary is the PostgreSQL adapter. Without it, Django cannot connect to PostgreSQL.

3.2 config/settings/base.py

```
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.getenv('SECRET_KEY', 'unsafe-dev-key-change-me')

# Database — reads from environment OR falls back to SQLite
DATABASES = {
    'default': {
        'ENGINE': os.getenv('DB_ENGINE', 'django.db.backends.sqlite3'),
        'NAME': os.getenv('DB_NAME', BASE_DIR / 'db.sqlite3'),
        'USER': os.getenv('DB_USER', ''),
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        'HOST': os.getenv('DB_HOST', ''),
        'PORT': os.getenv('DB_PORT', ''),
    }
}
```

CRITICAL: load_dotenv() MUST be called BEFORE DATABASES is defined. Otherwise, environment variables won't be read.

3.3 config/settings/local.py

```
from .base import *

DEBUG = True
ALLOWED_HOSTS = ['*']
```

```
# Static files
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'

# DO NOT override DATABASES here — let base.py handle it
```

CRITICAL: If you override DATABASES in local.py, it will ignore the environment variables from Docker and use SQLite instead.

3.4 Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Install system dependencies for PostgreSQL
RUN apt-get update && apt-get install -y \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy and install requirements FIRST (for Docker layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy entire project
COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

CRITICAL: Install requirements BEFORE copying code. This leverages Docker layer caching — if requirements.txt doesn't change, pip install is skipped on rebuild.

3.5 docker-compose.yml (FINAL WORKING VERSION)

```
version: "3.9"

services:
  db:
    image: postgres:16-alpine
    container_name: teamsync_db
```

```

volumes:
  - teamsync_postgres_data:/var/lib/postgresql/data
environment:
  POSTGRES_DB: teamsync
  POSTGRES_USER: teamsync_user
  POSTGRES_PASSWORD: teamsync_password
ports:
  - "5433:5432"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U teamsync_user -d teamsync"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s

web:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: teamsync_web
  command: >
    sh -c "python manage.py wait_for_db &&
      python manage.py migrate &&
      python manage.py runserver 0.0.0.0:8000"
  volumes:
    - ./app
  ports:
    - "8000:8000"
  environment:
    SECRET_KEY: "django-insecure-dev-key-change-in-production"
    DB_ENGINE: django.db.backends.postgresql
    DB_NAME: teamsync
    DB_USER: teamsync_user
    DB_PASSWORD: teamsync_password
    DB_HOST: db
    DB_PORT: 5432
    DEBUG: "True"
    DJANGO_SETTINGS_MODULE: config.settings.local
  depends_on:
    db:
      condition: service_healthy

volumes:
  teamsync_postgres_data:

```

CRITICAL POINTS:

- `POSTGRES_DB: teamsync` — This tells PostgreSQL to create the teamsync database on FIRST initialization only.
- `start_period: 30s` — Gives PostgreSQL 30 seconds to finish init before healthcheck marks it as failed.

- depends_on: condition: service_healthy — Web container waits until DB healthcheck passes.
- Named volume teamsync_postgres_data — Persists data across container restarts. On Windows, named volumes are more reliable than bind mounts.

3.6 apps/core/management/commands/wait_for_db.py

```
import time
import os
from django.db import connections
from django.db.utils import OperationalError
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    # Django command that waits for the specific database to be available

    def handle(self, *args, **options):
        self.stdout.write('Waiting for database...')

        db_name = os.getenv('DB_NAME', 'teamsync')
        attempts = 0
        max_attempts = 30

        while attempts < max_attempts:
            try:
                connection = connections['default']
                connection.ensure_connection()
                with connection.cursor() as cursor:
                    cursor.execute("SELECT 1")
                self.stdout.write(self.style.SUCCESS(f'Database "{db_name}" is ready!'))
                return
            except OperationalError as e:
                self.stdout.write(f'Waiting for database "{db_name}"... ({e})')
                time.sleep(1)
                attempts += 1

        self.stdout.write(self.style.ERROR(f'Database "{db_name}" failed after {max_attempts}
↪ attempts'))
        exit(1)
```

CRITICAL: Uses ensure_connection() + real SELECT 1 query. Just calling connections['default'] doesn't actually test the connection.

3.7 .dockerignore

```
.git
.gitignore
```

```
__pycache__
*.py[cod]
*.pyo
*.pyd
.Python
*.egg-info
dist
build

venv/
env/
ENV/

.vscode/
.idea/

.DS_Store
Thumbs.db

db.sqlite3
*.sqlite3

.env

staticfiles/
```

CRITICAL: Must include `db.sqlite3` — if this file exists on the host, it gets mounted into the container and Django may fall back to SQLite.

3.8 .gitignore

```
# Python
__pycache__/
*.py[cod]
*.pyo
*.pyd
.Python
*.egg-info/
.installed.cfg
*.egg

# Virtual Environments
venv/
ENV/
env/
.venv/

# Django
*.log
local_settings.py
```

```
db.sqlite3
db.sqlite3-journal
*.sqlite3
media/
staticfiles/

# Environment Variables / Secrets
.env
.env.local
.env.production

# IDE / Editors
.vscode/
.idea/
*.swp
*.swo
*~
.DS_Store
Thumbs.db

# Docker
*.pid

# Testing
.coverage
htmlcov/
.tox/
.pytest_cache/
nosetests.xml
coverage.xml
```

4. Running the Project

First Time Setup

```
# 1. Navigate to project root
cd C:\CODING    eamsync

# 2. Build and start everything
docker-compose up --build

# 3. In a NEW terminal, create superuser
docker-compose exec web python manage.py createsuperuser

# 4. Test the API
curl.exe -X POST http://127.0.0.1:8000/api/token/ `
  -H "Content-Type: application/json" `
  -d '{"email":"your@email.com","password":"yourpassword"}'
```

Subsequent Runs

```
docker-compose up
```

Stop Everything

```
docker-compose down
```

Complete Reset (Wipe Database)

```
docker-compose down
docker volume rm teamsync_postgres_data
docker-compose up --build
```

5. All Issues Faced & Solutions

Issue 1: `ModuleNotFoundError: No module named 'corsheaders'`

Symptom: Container crashes on startup with `ModuleNotFoundError`.

Root Cause: `django-cors-headers` not in `requirements.txt` or not installed in container.

Solution:

- Add `django-cors-headers>=4.3` to `requirements.txt`
- Rebuild with `docker-compose up --build` (not just `up`)

Issue 2: `PermissionError: [Errno 13] Permission denied: '/app/staticfiles/admin'`

Symptom: `collectstatic` fails with permission denied.

Root Cause: Non-root user (`appuser`) in `Dockerfile` cannot write to volume-mounted directory.

Solution: Remove `USER appuser` from `Dockerfile`. Run container as root for development.

```
# REMOVED:
# RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
# USER appuser
```

Issue 3: `AssertionError: The .create() method does not support writable nested fields by default`

Symptom: POST request to create `workspace/board` fails with `AssertionError`.

Root Cause: Serializers have nested serializers (e.g., `members = MemberSerializer(many=True)`) that are writable by default.

Solution: Make nested fields `read_only=True`:


```
class BoardDetailSerializer(serializers.ModelSerializer):
    lists = ListSerializer(many=True, read_only=True) # read_only=True
    workspace = serializers.PrimaryKeyRelatedField(read_only=True)
```

Issue 4: AssertionError: The field ‘member’ was declared on serializer WorkspaceDetailSerializer, but has not been included in the ‘fields’ option

Symptom: Serializer crashes because declared field name doesn’t match Meta.fields.

Root Cause: Field declared as member (singular) but Meta.fields has members (plural).

Solution: Ensure field names match exactly:

```
class WorkspaceDetailSerializer(serializers.ModelSerializer):
    members = MemberSerializer(many=True, read_only=True) # matches Meta.fields

    class Meta:
        fields = ['id', 'name', 'members'] # 'members' not 'member'
```

Issue 5: POST not allowed on /api/workspaces/

Symptom: 405 Method Not Allowed when trying to create workspace.

Root Cause: View uses ListAPIView which only supports GET.

Solution: Change to ListCreateAPIView:

```
class WorkspaceListView(generics.ListCreateAPIView): # was ListAPIView
```

Issue 6: name field is required when creating board

Symptom: POST to /api/boards/ returns 400 Bad Request with “name field is required”.

Root Cause: Frontend sends title but backend model/serializer expects name.

Solution: Match field names between frontend and backend. If model uses name, frontend must send name.

Issue 7: DRF Pagination — Frontend sees no data

Symptom: Workspaces/boards don’t appear in frontend. Empty state shown.

Root Cause: DRF returns {count: X, results: [...]} but frontend expects plain array.

Solution: Unwrap pagination in API client:

```
if (data && typeof data === 'object' && Array.isArray(data.results)) {
    return data.results;
```

```
}
```

Issue 8: FATAL: database “teamsync” does not exist (THE BIG ONE)

Symptoms:

- FATAL: database “teamsync” does not exist repeats infinitely
- PostgreSQL Database directory appears to contain a database; Skipping initialization
- Django wait_for_db fails after 30 attempts

Root Causes & Solutions:

Sub-Issue	Cause	Solution
Volume persists old data	pgdata folder wasn't actually deleted	Use docker volume rm teamsync_postgres_data
POSTGRES_DB not set initially	Volume created before env var added	Delete volume, set POSTGRES_DB: teamsync, restart
Bind mount on Windows unreliable	./pgdata:/var/lib/postgresql/data fails	Use named volume instead: teamsync_postgres_data:/var/lib/postgresql/data
Healthcheck too aggressive	PostgreSQL still initializing when healthcheck runs	Add start_period: 30s to healthcheck
wait_for_db connects to wrong DB	cursor() doesn't test actual database	Use ensure_connection() + SELECT 1

Complete Fix Sequence:

```
# 1. Stop everything
docker-compose down

# 2. Delete the old volume (CRITICAL)
docker volume rm teamsync_postgres_data

# 3. Verify no volumes exist
docker volume ls

# 4. Update docker-compose.yml:
#   - Use named volume (not bind mount)
#   - Set POSTGRES_DB: teamsync
#   - Add start_period: 30s to healthcheck

# 5. Start fresh
docker-compose up --build
```

Issue 9: Django uses SQLite instead of PostgreSQL

Symptom: `sqlite3.OperationalError` or `current_database()` function not found.

Root Causes:

1. `db.sqlite3` file exists on host and gets mounted into container
2. `local.py` overrides `DATABASES`
3. Environment variables not reaching Django

Solutions:

1. Delete `db.sqlite3`: `rm db.sqlite3`
 2. Remove `DATABASES` from `local.py`
 3. Verify env vars: `docker-compose exec web env | findstr DB_`
-

Issue 10: ModuleNotFoundError: No module named 'apps.core'

Symptom: Django can't find `wait_for_db` command.

Root Cause: `apps.core` not in `INSTALLED_APPS`.

Solution: Add to `base.py`:

```
INSTALLED_APPS = [  
    # ... other apps ...  
    'apps.core',  
]
```

6. Common Commands

```
# Start everything  
docker-compose up --build  
  
# Start in background  
docker-compose up -d  
  
# View logs  
docker-compose logs -f web  
docker-compose logs -f db  
  
# Run management commands  
docker-compose exec web python manage.py migrate  
docker-compose exec web python manage.py makemigrations  
docker-compose exec web python manage.py createsuperuser  
docker-compose exec web python manage.py test  
docker-compose exec web python manage.py shell  
  
# Database shell  
docker-compose exec db psql -U teamsync_user -d teamsync
```

```
# Stop
docker-compose down

# Stop and wipe database
docker-compose down -v

# List volumes
docker volume ls

# Delete specific volume
docker volume rm teamsync_postgres_data

# Nuclear option (delete everything Docker has)
docker system prune -a --volumes -f
```

7. For Future AI Agents / Developers

If User Says: “Database does not exist”

1. Check if named volume exists: `docker volume ls`
2. If `teamsync_postgres_data` exists, it was created WITHOUT `POSTGRES_DB`
3. MUST DELETE VOLUME: `docker volume rm teamsync_postgres_data`
4. Verify `docker-compose.yml` has:
 - `POSTGRES_DB: teamsync`
 - Named volume (not bind mount)
 - `start_period: 30s` in healthcheck
5. Run `docker-compose up --build`

If User Says: “SQLite instead of PostgreSQL”

1. Check: `docker-compose exec web env | findstr DB_ENGINE`
2. Should show: `DB_ENGINE=django.db.backends.postgresql`
3. If not, check `local.py` doesn't override `DATABASES`
4. Delete `db.sqlite3` from host machine
5. Verify `base.py` has `load_dotenv()` before `DATABASES`

If User Says: “Permission denied on staticfiles”

1. Remove USER `appuser` from `Dockerfile`
2. Or add `RUN mkdir -p /app/staticfiles && chown -R appuser:appuser /app/staticfiles`

If User Says: “POST not allowed”

1. Check view class: must be `ListCreateAPIView`, not `ListAPIView`
2. Check `permission_classes` includes `IsAuthenticated`

Key Insight: PostgreSQL Init Behavior

PostgreSQL ONLY creates database from POSTGRES_DB on FIRST initialization. If volume has ANY existing data, init is SKIPPED entirely. This is why deleting the volume is ALWAYS the fix for “database does not exist”.

Windows-Specific Notes

- Bind mounts (./folder:/container/path) are unreliable on Windows Docker Desktop
- Always use named volumes for PostgreSQL on Windows
- `rm -rf` may not work in PowerShell — use `Remove-Item -Recurse -Force`
- Docker volumes may persist in WSL even after Windows deletion — use `docker volume rm`

Golden Rules

1. Named volumes > bind mounts for databases on Windows
2. Delete volume when changing POSTGRES_DB or any init config
3. `--build` flag is required when `requirements.txt` changes
4. `db.sqlite3` must be deleted when switching to PostgreSQL
5. `load_dotenv()` before DATABASES in `base.py`
6. `local.py` must NOT override DATABASES

End of Document — TeamSync Docker Troubleshooting Guide